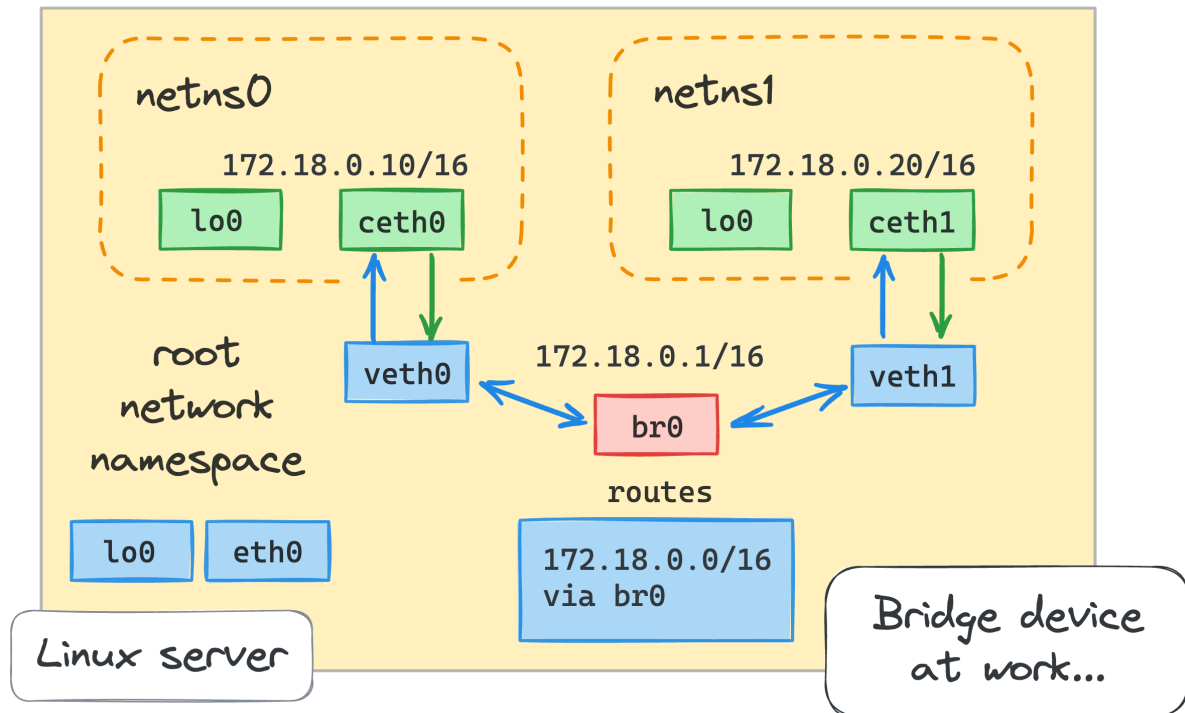


1주차

리눅스 네임 스페이스 네트워크 통신

<https://labs.iximiuz.com/tutorials/container-networking-from-scratch>

실습 환경 : Ubuntu 22.04



루트 네임스페이스에서 netns0 통신 과정

이제 루트에서 172.18.0.10(netns0)으로 핑을 보냄

1. 호스트의 라우팅 테이블 확인

```
default via 172.16.0.1 dev eth0
172.16.0.0/24 dev eth0 proto kernel scope link src 172.16.0.2
172.18.0.0/16 dev br0 proto kernel scope link src 172.18.0.1
```

172.18.x.x이므로 br0 인터페이스를 통해 전달합니다

2. 이제 172.18.0.10이 누군지 확인해야함, L2 레벨로 패킷을 보내기 위해 상대방의 하드웨어 주소(MAC)을 알아내야 함
 - ARP Request를 br0 인터페이스를 통해 broadcast 전달 (ARP)

- br0은 자신에게 쫓힌 모든 랜선 포트 (veth0, veth1)로 전달
 - netns0 안에 있는 ceth0 랜카드가 답장을 보냄 (ARP Reply)
 - 호스트는 이 주소를 메모장 (ip neigh)에 기록
3. br0을 통한 패킷 배달 - 이제 호스트는 핑 패킷에 목적지 MAC주소를 적어서 br0에 던짐
- br0은 패킷의 MAC 주소를 보고 veth0포트에 연결되어 있는거 확인하고 veth0 랜선으로 패킷을 밀어 넣어줌
 - 랜선을 타고 패킷이 netns0내부의 ceth0(172.18.0.10) 랜카드에 도착

답장 보내기 netns→호스트(172.18.0.1)

1. netns0 내부의 라우팅 테이블을 봄

```
root@ubuntu-01:~# nsenter --net=/run/netns/netns0 ip route list
default via 172.18.0.1 dev ceth0
172.18.0.0/16 dev ceth0 proto kernel scope link src 172.18.0.10
root@ubuntu-01:~# ip netns identify
```

목적지가 172.18.0.1 이네 라우팅 규칙에 있으니까 ceth0으로 내보내면 되겠다

2. br0을 거쳐 호스트로 최종 배달

- 패킷이 ceth0을 타고 나와 가상 랜선 쌍의 반대편인 호스트 쪽 veth0으로 나옴
- veth0은 br0에 연결 되었으므로 패킷이 br0으로 들어감
- br0은 목적지 IP가 (172.18.0.1)임으로, 호스트 OS의 네트워크 스택(CPU)으로 패킷 전달
- 호스트 터미널 창에 성공 메시지 출력

```
root@ubuntu-01:~# ping -c 2 172.18.0.10
PING 172.18.0.10 (172.18.0.10) 56(84) bytes of data.
64 bytes from 172.18.0.10: icmp_seq=1 ttl=64 time=0.028 ms
64 bytes from 172.18.0.10: icmp_seq=2 ttl=64 time=0.043 ms

--- 172.18.0.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1009ms
rtt min/avg/max/mdev = 0.028/0.035/0.043/0.007 ms
```

확인해볼점

1. 브릿지의 역할

- 위에 실습에서 브릿지가 없을 때에는 호스트에 동일한 `172.18.0.0/16` 네트워크에 대해 여러 인터페이스가 존재합니다.
- 호스트가 `172.18.x.x`로 통신을 보내게 된다면 먼저 작성되어진 라우팅 테이블을 따라가는 특성때문에 `netns1`의 ip인 `172.18.0.21`으로 보내게 된다면 호스트의 먼저 작성된 라우팅 테이블이 먼저 응답하기 때문에 `172.18.x.x`는 `netns0`으로 가게 됩니다.

```
root@ubuntu-01:~# ip route list
default via 172.16.0.1 dev eth0
172.16.0.0/24 dev eth0 proto kernel scope link src 172.16.0.2
172.18.0.0/16 dev veth0 proto kernel scope link src 172.18.0.11
172.18.0.0/16 dev veth1 proto kernel scope link src 172.18.0.21
```

- 이러한 라우팅 충돌을 없애고자 리눅스는 가상화된 네트워크 기능인 `bridge` 제공합니다.
네트워크 스위치 처럼 작동해 연결된 장치의 IP 주소는 신경 쓰지 않고 L2레벨에서 작동합니다.

결론적으로 리눅스에서 브릿지는 여러 네트워크 인터페이스를 하나의 L2 네트워크로 연결하며, Root Namespace에서 `172.18.0.0/16 → br0` 과 같이 하나의 라우팅으로 간편하게 합니다. 이후 실제 라우팅은 MAC 주소를 기반으로 결정합니다.

IP를 보고 어디로 내보낼지는 Routing이 결정하고, 같은 L2 네트워크에서 IP에 해당하는 MAC을 알아내는 것은 ARP, 실제 패킷을 어느 포트로 전달할지는 Bridge가 결정합니다.

ARP

ARP(Address Resolution Protocol)은 IP 주소를 MAC 주소와 매칭시키기 위한 프로토콜

IP 주소와 MAC 주소를 일대일 매칭하여 LAN(Layer 2) 에서 목적지를 제대로 찾아갈 수 있도록 돕습니다. IP 주소와 MAC 주소를 일대일 대응하여 테이블로 정리하고(ARP Table) 목적지 IP에 맞는 목적지 MAC 주소를 전달

과정은 간단하게 호스트가 ARP Request를 보냅니다. 그걸 스위치가 받고 broadcast 그러면 그걸 받은 PC들이 ip를 확인하고 그 IP가 본인 것이라면 ARP Reply 를 보냅니다. 그걸 다시 스위치가 호스트에게로 전달

ARP로 구하는 MAC 주소는 항상 최종 목적지의 MAC 주소인 것은 아니고, 현재 L2 구간에서 패킷을 전달할 다음 장치의 MAC 주소를 알아내는 데 사용됩니다. (예 구글로 통

신할때 구글의 MAC주소를 알지는 않는 것)

상세 그림과 설명 패킷 분석은 아래 사이트를 확인하세요

[https://m.blog.naver.com/PostView.naver?
isHttpsRedirect=true&blogId=sujunghan726&logNo=220322717607](https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=sujunghan726&logNo=220322717607)

질문 2가지

1. CNI 플러그인은 Pod 생성시 이러한 Linux 네트워크 구성을 자동화한다고 하는데 구체적으로 어떤것들을 자동화하는지 궁금합니다.
2. Namespaces는 어떤거를 격리하는지 궁금합니다.
<https://wikidocs.net/336973>